

Helix OTA — 1.0.0-MVP REST API (/api/v1) Endpoint Specification

Field	Value
Revision	2
Created	2026-06-07
Last modified	2026-06-07 (rev 2)
Status	active
Status summary	Full prose specification of every /api/v1 REST endpoint for the Helix OTA 1.0.0-MVP control plane: authentication, device registry, artifact intake, releases, deployments (all-targets for MVP), the device update-check + telemetry surface, and the cross-cutting concerns (OAuth2/JWT + RBAC, rate limiting, Brotli/HTTP3 negotiation, error model). Companion machine-readable contract is openapi.yaml. HelixConstitution clause numbers (§11.4.61, §7.1, §11.4.6, §11.4.123, §11.4.74, §11.4.28) are carried from corpus convention and are UNVERIFIED against the authoritative Constitution text. The exact AOSP payload_properties.txt optional-header set (SWITCH_SLOT_ON_REBOOT, RUN_POST_INSTALL, DISABLE_DOWNLOAD_RESUME) on Android 15 is UNVERIFIED (carried from ADR-0004 §6 / aosp-update-engine open items). Range/HTTP-3 semantics for the artifact path are UNVERIFIED pending the ADR-0004 §6 spike. Catalogue-submodule public surfaces (auth, ratelimiter, middleware, http3, Storage) are reused but their exact API was not inspected in this revision (UNVERIFIED).
Issues	Rev 2: gave telemetry its own 6-value TelemetryEventType enum (no idle) distinct from the 7-value UpdateState
Fixed	

Field	Value
Continuation	and made §8.2/§12.2/openapi agree; added explicit own-deviceId resource-ownership notes to /client/update, /client/telemetry, /devices/{deviceId}/status; added RateLimit-Limit/-Remaining/-Reset response headers to the rate-limited routes in openapi; documented Vary: Accept-Encoding as a global JSON-response convention (resolving the login-only omission); modelled the /devices/register provisioning-token alternative as a second provisioningToken security scheme. Confirm the auth brick exposes OAuth2 password + refresh + RBAC as specified (§4); close the ADR-0004 §6 transport spikes (Range over HTTP/3, Android-15 resume headers); finalize concrete rate-limit numbers from MVP load tests; add per-endpoint OpenTelemetry span names; bind the device-identity-token-to-hardware-id option if security/Security-KMP lacks it (per submodule_reuse_map.md §5).
Owner	Helix OTA control-plane team
Related	openapi.yaml ; ../.. /research/adr/adr-0003-server-topology.md ; ../.. /research/adr/adr-0004-transport.md ; ../.. /research/adr/adr-0002-supply-chain-trust.md ; ../.. /00-master/submodule_reuse_map.md

Table of contents

1. [Purpose and scope](#)
2. [Conventions](#)
3. [Transport, negotiation, and compression](#)
4. [Authentication, authorization \(OAuth2/JWT + RBAC\)](#)
5. [Rate limiting](#)
6. [Error model](#)
7. [Auth endpoints](#)
 - [7.1 POST /api/v1/auth/login](#)
 - [7.2 POST /api/v1/auth/refresh](#)
8. [Device endpoints](#)
 - [8.1 POST /api/v1/devices/register](#)
 - [8.2 GET /api/v1/devices/{deviceId}/status](#)

9. Artifact endpoints
 - 9.1 POST /api/v1/artifacts/upload
 - 9.2 GET /api/v1/artifacts/{artifactId}
 10. Release endpoints
 - 10.1 POST /api/v1/releases
 - 10.2 GET /api/v1/releases / GET /api/v1/releases/{releaseId}
 11. Deployment endpoints (all-targets for MVP)
 - 11.1 POST /api/v1/deployments
 - 11.2 GET /api/v1/deployments/{deploymentId}
 12. Client (device) endpoints
 - 12.1 GET /api/v1/client/update — update check (204 / 200)
 - 12.2 POST /api/v1/client/telemetry — telemetry report
 13. Status code summary
 14. Testing (four-layer)
 15. Catalogue-first reuse and decoupling
 16. Compliance notes (HelixConstitution)
-

1. Purpose and scope

This document specifies, in prose, the complete public REST surface of the Helix OTA **1.0.0-MVP** control plane. The control plane is a **modular monolith** — one deployable Go binary with enforced internal seams (ADR-0003) — built on the **LOCKED stack**: Go + Gin (gin-gonic) + Brotli + HTTP/3 (QUIC) via the `vasic-digital/http3` submodule, with automatic HTTP/2 + gzip fallback. **REST /api/v1 is the primary/compatibility surface; gRPC is optional/internal only and is out of scope for this document** (ADR-0004 §4, contradiction C4).

Scope of MVP per the master design §6: Auth, Artifact intake + validation, Device registry, Release/Deploy (**all-targets** — the staged-rollout engine lands in 1.0.1), Telemetry ingest, and the Android-agent-facing device endpoints. The trust model for MVP is **per-artifact SHA-256 + detached signature + AVB** verified server-side on upload and device-side before apply; **TUF/Uptane device-side enforcement is deferred to 1.0.1+** (ADR-0002). Signing interfaces are designed MVP-forward so TUF drops in without rework: every artifact is treated as an **opaque target identified by path + length + SHA-256** (ADR-0002 §4.2).

Two endpoint classes with **opposite transport requirements** are described (ADR-0004 §1):

- **Control-plane endpoints** (all JSON /api/v1/* below) — many small request/response pairs; Brotli/gzip compression applies; HTTP/3 connection migration benefits the 15-min-jitter poll profile.
- **Artifact-download path** (the actual `payload.bin` bytes) — large, opaque, already-compressed binary served **byte-identical, ZIP_STORED, content-encoding identity, with mandatory HTTP Range support**. This path is **not** a JSON endpoint; the control plane returns a *reference* to it from the update-check (§12.1). Content compression **MUST NOT** be applied to it (ADR-0004 §4, the load-bearing rule).

2. Conventions

- **Base path:** `/api/v1`. All paths below are relative to the server origin.
- **Media type:** `application/json; charset=utf-8` for all request/response bodies except the artifact bytes (binary) and multipart upload (§9.1).
- **Identifiers:** opaque server-issued strings. The OpenAPI schema models them as `format: uuid` where the control plane mints UUIDs; clients **MUST** treat them as opaque.
- **Timestamps:** RFC 3339 / ISO-8601 UTC strings (`2026-06-07T00:00:00Z`).
- **Versions:** semantic-version strings (`1.2.3`); version monotonicity is enforced on release creation (master §6 validation pipeline).
- **Idempotency:** `POST /api/v1/devices/register` and `POST /api/v1/deployments` accept an optional `Idempotency-Key` request header; a repeated key returns the original result rather than creating a duplicate (UNVERIFIED that the middleware brick supplies this — wire if present, otherwise implement in the route).
- **Pagination:** list endpoints accept `?limit` (default 50, max 200) and `?cursor` (opaque); responses carry `next_cursor` (null when exhausted).
- **Auth header:** Authorization: Bearer <JWT> for human/admin and device-bearer calls (§4).
- **Correlation:** every response carries `X-Request-Id`; clients **SHOULD** echo it when reporting problems. Tracing via the observability brick (OpenTelemetry).

3. Transport, negotiation, and compression

Honors the LOCKED stack and ADR-0004:

1. **Protocol negotiation order at the edge:** try **HTTP/3 (QUIC, UDP/443)** via the `http3` submodule → fall back to **HTTP/2 (TCP, TLS 1.3)** automatically when QUIC/UDP-443 is unreachable. TLS 1.3 throughout. HTTP/3 advertised via the `Alt-Svc` response header on the HTTP/2 path (standard QUIC discovery mechanism).
2. **Content compression — two-class rule:**
 - **Control-plane JSON** (every endpoint in §7–§12.2): negotiate via the request `Accept-Encoding` header in order **br (Brotli)** → **gzip** → **identity**. The chosen encoding is echoed in `Content-Encoding`; `Vary: Accept-Encoding` is always set. Brotli is primary; gzip is the fallback for older clients. (Brotli quality/parameter tuning is UNVERIFIED — benchmark per ADR-0004 §6.)
 - **Artifact bytes** (the `payload.bin` download referenced from §12.1): **always Content-Encoding: identity**, served `ZIP_STORED` and byte-identical, with `Accept-Ranges: bytes` and full **HTTP Range** request support so the device's `update_engine` can range-fetch by offset/size. Brotli/gzip **MUST NOT** be applied here — doing so would break the `ZIP_STORED` offset/length contract and the device's `FILE_HASH/METADATA_HASH` verification. A CI/serving guard enforces this (ADR-0004 §5.2).
3. **Compression is handled by the middleware brick** (Brotli/gzip negotiation), the `http3` brick (protocol), and the `ratelimiter/recovery` bricks — no hand-rolled transport (`submodule_reuse_map.md` Transport row).

Clients SHOULD send `Accept-Encoding: br, gzip` on control-plane calls. The device update-check (§12.1) is itself a small JSON response and participates in Brotli/gzip negotiation; only the *artifact it points to* is identity.

4. Authentication, authorization (OAuth2/JWT + RBAC)

Authentication and RBAC are **reused from the auth + security + middleware catalogue bricks** (KMP: Auth-KMP, Security-KMP); Helix OTA only wires policies and routes — no auth logic is re-implemented (`submodule_reuse_map.md` Auth row). (UNVERIFIED: that auth already exposes OAuth2/JWT + RBAC exactly as specified here.)

4.1 Token model (OAuth2 / JWT)

- **Grant types (MVP):** OAuth2 **Resource Owner Password Credentials** for the admin/dashboard login (§7.1) and **Refresh Token** for silent renewal (§7.2). These are the only grants in MVP; authorization-code/PKCE for third-party clients is out of scope.
- **Access token:** short-lived **JWT** (default TTL 15 minutes, configurable via the config brick) carrying `sub` (subject id), `roles` (array), `scope`, `iat`, `exp`, and `iss`. Signed by the security brick (asymmetric; key custody per ADR-0002 signer abstraction). The resource server verifies signature + expiry on every request.
- **Refresh token:** long-lived (default 30 days, configurable), opaque, server-side revocable; rotated on each use (a used refresh token is invalidated when a new pair is issued).
- **Device bearer:** registered devices authenticate the client endpoints (§12) with a **device-scoped bearer JWT** issued at registration (§8.1), carrying `sub=<deviceId>` and `roles=["device"]`. (Binding the device token to a hardware id via Android KeyStore is an extend item on security/Security-KMP if not already present — UNVERIFIED, see `submodule_reuse_map.md` §5.)

4.2 RBAC roles and route policy

Role	Capabilities
admin	Full control: manage artifacts, releases, deployments, devices, and read all status/telemetry.
operator	Create/read releases and deployments; read devices/telemetry; cannot manage users or API keys.
viewer	Read-only: list/read releases, deployments, devices, status.
device	Only the client endpoints: update-check (§12.1) for its own <code>deviceId</code> , and telemetry report (§12.2) for itself. Cannot access admin/operator routes.

Route → minimum role:

Route	Min role
POST /auth/login, POST /auth/refresh	none (public)
POST /devices/register	operator (provisioning) or a valid provisioning token; the response mints the device's own device token
GET /devices/{id}/status	viewer
POST /artifacts/upload	operator
GET /artifacts/{id}	viewer
POST /releases, GET /releases*	operator (create), viewer (read)
POST /deployments	operator
GET /deployments/{id}	viewer
GET /client/update	device (own id only)
POST /client/telemetry	device (own id only)

Authorization failures: missing/invalid token → **401**; valid token lacking the required role or accessing another device's resource → **403**.

5. Rate limiting

Rate limiting is **reused from the ratelimiter brick** wired in middleware (submodule_reuse_map.md Transport row). Limits are per-principal (token sub) and, for unauthenticated routes, per source IP.

- **Algorithm:** token-bucket (sliding window), configurable via the config brick.
- **Response on breach: 429 Too Many Requests** with the standard error envelope (§6) and a Retry-After header (seconds).
- **Advisory headers** on every rate-limited route: RateLimit-Limit, RateLimit-Remaining, RateLimit-Reset (draft IETF RateLimit header fields).
- **Tiered defaults (configurable; concrete numbers are UNVERIFIED until set from MVP load tests — ADR-0003 §3.2 / continuation):**
 - POST /auth/login: strict (brute-force protection), e.g. low per-IP burst.
 - GET /client/update, POST /client/telemetry: tuned for the **15 min + jitter** fleet poll cadence (master §6 / D7) — generous steady-state, burst-tolerant for jitter.
 - Admin/operator write routes: moderate.
- The artifact-download path is range-served and is **not** subject to the JSON-route limiter; abuse protection there is connection/bandwidth-level (out of scope here).

6. Error model

All non-2xx responses (except 204 No Content and the artifact-byte path) return a single, consistent JSON envelope (Error schema in the OpenAPI):

```
{
  "error": {
    "code": "VALIDATION_FAILED",
    "message": "human-readable summary, never a secret or stack
      trace",
    "request_id": "01J...",
    "details": [
      { "field": "version", "issue": "must be greater than the
        latest release version" }
    ]
  }
}
```

- **code** — stable machine-readable token (SCREAMING_SNAKE_CASE). Enumerated set: UNAUTHENTICATED, FORBIDDEN, NOT_FOUND, VALIDATION_FAILED, CONFLICT, UNSUPPORTED_MEDIA_TYPE, PAYLOAD_TOO_LARGE, RATE_LIMITED, SIGNATURE_INVALID, HASH_MISMATCH, VERSION_NOT_MONOTONIC, INTERNAL.
- **message** — safe human text; never leaks stack traces, secrets, or internal paths (handled by the recovery brick which converts panics into 500 INTERNAL without disclosure).
- **request_id** — mirrors the X-Request-Id response header for correlation.
- **details** — optional array of field-level problems (used by VALIDATION_FAILED).

The artifact-byte path uses bare HTTP status semantics (200/206/404/416/501) without the JSON envelope, since the consumer is `update_engine`, not a JSON client.

7. Auth endpoints

7.1 POST /api/v1/auth/login

Exchanges username + password for an access/refresh token pair (OAuth2 ROPC).

- **Auth:** none (public). Strict rate limit (§5).
- **Request body** (LoginRequest):

```
{ "username": "admin@example.com", "password": "....." }
```

- **Response 200** (TokenResponse):

```
{
  "access_token": "<JWT>",
  "token_type": "Bearer",
  "expires_in": 900,
  "refresh_token": "<opaque>",
  "roles": ["admin"]
}
```

- **Status codes:** 200 OK; 400 VALIDATION_FAILED (missing field); 401 UNAUTHENTICATED (bad credentials); 429 RATE_LIMITED.

7.2 POST /api/v1/auth/refresh

Rotates a refresh token into a new access/refresh pair.

- **Auth:** the refresh token itself (in body); no access token required.
- **Request body** (RefreshRequest):

```
{ "refresh_token": "<opaque>" }
```

- **Response 200:** TokenResponse (same shape as §7.1); the old refresh token is invalidated (rotation).
- **Status codes:** 200 OK; 400 VALIDATION_FAILED; 401 UNAUTHENTICATED (expired, revoked, or already-rotated token); 429 RATE_LIMITED.

8. Device endpoints

8.1 POST /api/v1/devices/register

Provisions a device in the registry and mints its device-scoped bearer token. Persisted via the database brick; wire types from ota-protocol (submodule_reuse_map.md Device registry row).

- **Auth:** operator (or a valid provisioning token). Accepts optional Idempotency-Key.
- **Request body** (DeviceRegistration):

```
{  
  "hardware_id": "rk3588-orangepi5max-AABBCCDD",  
  "model": "OrangePi5Max",  
  "os": "android",  
  "os_version": "15",  
  "current_version": "1.0.0",  
  "group": "field-fleet-a",  
  "metadata": { "region": "eu-west" }  
}
```

- **Response 201** (DeviceRegistered):

```
{  
  "device_id": "8f3a...uuid",  
  "hardware_id": "rk3588-orangepi5max-AABBCCDD",  
  "device_token": "<device-scoped JWT>",  
  "token_type": "Bearer",  
  "expires_in": 86400,  
  "registered_at": "2026-06-07T00:00:00Z"  
}
```

- **Status codes:** 201 Created; 200 OK (idempotent replay of same Idempotency-Key); 400 VALIDATION_FAILED; 401/403; 409 CONFLICT (hardware_id already registered with a different identity); 429.

8.2 GET /api/v1/devices/{deviceId}/status

Returns the current registry + last-known runtime status of a device.

- **Auth:** viewer (admin/operator/viewer). A device token may read **only its own** id.
- **Response 200** (DeviceStatus):

```
{
  "device_id": "8f3a...uuid",
  "hardware_id": "rk3588-orangepi5max-AABBCCDD",
  "current_version": "1.0.0",
  "target_version": "1.1.0",
  "last_seen": "2026-06-07T00:14:00Z",
  "update_state": "idle",
  "active_slot": "a",
  "health": { "ok": true, "last_error_code": null }
}
```

update_state enumerates the device lifecycle (master §9): idle, download_started, installing, installed, verifying, success, failure (7 values; UpdateState schema). The telemetry event set (§12.2) is the same lifecycle **minus idle** (an idle device emits no event), so it is a distinct 6-value enum (TelemetryEventType schema) — the two are intentionally not the same type.

- **Status codes:** 200 OK; 401/403; 404 NOT_FOUND.

9. Artifact endpoints

9.1 POST /api/v1/artifacts/upload

Uploads an OTA artifact blob and runs the **server-side validation pipeline**: structure → SHA-256 hash → detached-signature verification → (deferred to release: version monotonicity, target compatibility) (master §6; ota-artifact-validator + security + Storage). Blob persisted to MinIO/S3 via the Storage brick; the artifact is stored **ZIP_STORED** so it can later be range-served byte-identically (ADR-0004 §4).

- **Auth:** operator.
- **Request:** multipart/form-data with parts:
 - file — the OTA .zip / payload.bin package (binary; ZIP_STORED).
 - metadata — JSON part (ArtifactUploadMetadata): declared sha256 (lowercase hex), signature (base64 detached signature), version, os, target_model, and the AOSP streaming fields file_hash, file_size, metadata_hash, metadata_size, payload_offset, payload_size (sourced from payload_properties.txt; the optional header set on Android 15 is UNVERIFIED — aosp-update-engine open items).
- **Server checks (reject on any failure):**
 - Computed SHA-256 of the stored bytes **MUST** equal the declared sha256 → else 422 HASH_MISMATCH.
 - Detached signature **MUST** verify against the declared sha256/bytes using the trusted signing key → else 422 SIGNATURE_INVALID.

- Media type / structure sane → else 415 UNSUPPORTED_MEDIA_TYPE or 400 VALIDATION_FAILED.
- Size within the configured cap → else 413 PAYLOAD_TOO_LARGE.
- **Response 201** (Artifact):

```
{
  "artifact_id": "a91c...uuid",
  "sha256": "9f86d0...",
  "size": 379074366,
  "os": "android",
  "target_model": "OrangePi5Max",
  "version": "1.1.0",
  "storage_ref": "s3://helix-artifacts/a91c...",
  "verified": true,
  "uploaded_at": "2026-06-07T00:00:00Z"
}
```

- **Status codes:** 201 Created; 400 VALIDATION_FAILED; 401/403; 413 PAYLOAD_TOO_LARGE; 415 UNSUPPORTED_MEDIA_TYPE; 422 HASH_MISMATCH / 422 SIGNATURE_INVALID; 429.

9.2 GET /api/v1/artifacts/{artifactId}

Returns artifact **metadata** (not the bytes). The actual bytes are fetched by the device from the URL returned in the update-check (§12.1), which is the only Range-served, identity-encoded path.

- **Auth:** viewer.
- **Response 200:** Artifact (same schema as §9.1 response).
- **Status codes:** 200 OK; 401/403; 404 NOT_FOUND.

The byte-stream download endpoint (referenced, not a JSON API) responds with 200 OK (full) or 206 Partial Content (Range), Accept-Ranges: bytes, Content-Encoding: identity; 404 if missing; 416 Range Not Satisfiable for a bad range; 501 if Range is unsupported by the backend (which MUST NOT happen in MVP — guarded). This path is documented here for completeness; its exact wiring is the Storage/object-store concern (ADR-0004 §4).

10. Release endpoints

A **release** binds a validated artifact to a published, deployable version. Creation enforces **version monotonicity** and **target compatibility** (master §6).

10.1 POST /api/v1/releases

- **Auth:** operator.
- **Request body** (ReleaseCreate):

```
{
  "artifact_id": "a91c...uuid",
```

```

"version": "1.1.0",
"os": "android",
"target_model": "OrangePi5Max",
"notes": "Security patch + telemetry fix",
"min_current_version": "1.0.0"
}

```

- **Server checks:** referenced artifact exists and `verified=true`; version strictly greater than the latest release for the same `os+target_model` → else 409 `VERSION_NOT_MONOTONIC`.
- **Response 201** (Release):

```

{
  "release_id": "r77e...uuid",
  "artifact_id": "a91c...uuid",
  "version": "1.1.0",
  "os": "android",
  "target_model": "OrangePi5Max",
  "status": "published",
  "created_at": "2026-06-07T00:00:00Z"
}

```

- **Status codes:** 201; 400 `VALIDATION_FAILED`; 401/403; 404 `NOT_FOUND` (artifact); 409 `CONFLICT` / 409 `VERSION_NOT_MONOTONIC`; 429.

10.2 GET /api/v1/releases / GET /api/v1/releases/{releaseId}

- **Auth:** viewer.
- **List** GET /releases — paginated (§2); optional filters `?os`, `?target_model`, `?status`. Response `ReleaseList { items: [Release], next_cursor }`.
- **Read one** GET /releases/{releaseId} — response `Release`; 404 `NOT_FOUND` if absent.
- **Status codes:** 200; 401/403; 404 (single).

11. Deployment endpoints (all-targets for MVP)

A **deployment** assigns a release to a target set. For MVP the only target strategy is **all-targets** (all devices matching the release `os+target_model`, optionally narrowed by group). Staged/percentage rollout is **deferred to 1.0.1** via the extractable `ota-rollout-engine` seam (ADR-0003 §3.1; master §6/§8). The MVP request schema reserves a `strategy` field whose only accepted value is `all-targets` so the staged engine drops in without a breaking change.

11.1 POST /api/v1/deployments

- **Auth:** operator. Accepts optional `Idempotency-Key`.
- **Request body** (`DeploymentCreate`):

```

{
  "release_id": "r77e...uuid",
  "strategy": "all-targets",

```

```
"group": "field-fleet-a"
}
```

strategy MUST be "all-targets" in MVP; any other value → 400
VALIDATION_FAILED (with a details entry noting staged rollout is 1.0.1+).

- **Response 201** (Deployment):

```
{
  "deployment_id": "d12b...uuid",
  "release_id": "r77e...uuid",
  "strategy": "all-targets",
  "group": "field-fleet-a",
  "status": "active",
  "target_count": 1240,
  "created_at": "2026-06-07T00:00:00Z"
}
```

- **Status codes:** 201; 200 (idempotent replay); 400 VALIDATION_FAILED; 401/403; 404 NOT_FOUND (release); 409 CONFLICT (an active deployment already targets the set); 429.

11.2 GET /api/v1/deployments/{deploymentId}

- **Auth:** viewer.
- **Response 200** (DeploymentStatus): the Deployment fields plus aggregate progress { pending, downloading, installed, succeeded, failed } counts derived from device telemetry (master §9).
- **Status codes:** 200; 401/403; 404 NOT_FOUND.

12. Client (device) endpoints

These are the Android-agent-facing endpoints (ota-android-agent). They use the device bearer token and may act **only on the calling device's own id** (§4.2).

12.1 GET /api/v1/client/update — update check (204 / 200)

The device polls this endpoint (default **15 min + jitter**, configurable via config/D7) to ask “is there an update for me?”. This is the heart of the device contract.

- **Auth:** device (own id). Query/identity: the calling deviceId is taken from the token sub; the agent MAY send ?current_version= to let the server short-circuit.
- **Two outcomes:**
 - **204 No Content** — the device is already on the target version (or no deployment applies). **No body.** This is the common steady-state response and keeps poll traffic cheap. (Brotli/gzip negotiation is moot for an empty body.)
 - **200 OK** with an UpdateAvailable body — an update is assigned. The body gives the device exactly what update_engine.applyPayload(url, offset, size, headers) needs (aosp-update-engine §6/§7):

```

{
  "release_id": "r77e...uuid",
  "version": "1.1.0",
  "url": "https://artifacts.helix.example/a91c....zip",
  "offset": 1234,
  "size": 379074366,
  "sha256": "9f86d0...",
  "signature": "BASE64-detached-signature",
  "payload_properties": {
    "FILE_HASH": "base64...",
    "FILE_SIZE": 379074366,
    "METADATA_HASH": "base64...",
    "METADATA_SIZE": 46866
  }
}

```

- **Field contract:**

- `url` — HTTPS location of the OTA package; **Range-served, Content-Encoding: identity, ZIP_STORED** (§3, §9.2). The device's `update_engine` byte-range-fetches `payload.bin` from `offset` for `size` bytes.
- `offset` / `size` — position and length of `payload.bin` **inside the uncompressed ZIP** (the `applyPayload` URL-form arguments).
- `sha256` — lowercase-hex SHA-256 of the artifact, used for the Helix verify-before-apply gate (the device fetches to a local verified file, checks `sha256` + `signature`, then applies — ADR-0002 §4.1 local-verified-file apply).
- `signature` — base64 detached signature over the artifact (MVP plain-signing trust; TUF metadata layers over this byte-identically in 1.0.1+ per ADR-0002).
- `payload_properties` — the AOSP streaming headers (`FILE_HASH`, `FILE_SIZE`, `METADATA_HASH`, `METADATA_SIZE`) passed straight through to `applyPayload`'s `keyValuePairHeaders`. The **optional** header set (`SWITCH_SLOT_ON_REBOOT`, `RUN_POST_INSTALL`, `DISABLE_DOWNLOAD_RESUME`) is **UNVERIFIED** on Android 15 (aosp-update-engine open items); Helix deliberately does **not** send `DISABLE_DOWNLOAD_RESUME` so native download-resume is not suppressed (ADR-0004 §4.4).
- **Status codes:** 200 OK (update available); 204 No Content (up to date); 401/403; 429 RATE_LIMITED.
- **Caching:** responses are Cache-Control: no-store (assignment is device-specific and changes with rollout/registry state).

12.2 POST /api/v1/client/telemetry — telemetry report

The device reports lifecycle events and health. Ingested via the observability brick using the `ota-telemetry-schema` codecs; events feed the dashboard and (in 1.0.1) the rollout halt/advance logic (master §9).

- **Auth:** device (own id). Tuned rate limit for the poll/report cadence (§5).
- **Request body** (`TelemetryReport`): a batch of events.

```

{
  "device_id": "8f3a...uuid",
  "deployment_id": "d12b...uuid",
  "events": [
    {
      "event": "download_started",
      "version": "1.1.0",
      "timestamp": "2026-06-07T00:15:00Z",
      "error_code": null,
      "detail": null
    },
    {
      "event": "failure",
      "version": "1.1.0",
      "timestamp": "2026-06-07T00:20:00Z",
      "error_code": "PAYLOAD_VERIFICATION_FAILED",
      "detail": "FILE_HASH mismatch"
    }
  ],
  "health": { "battery_pct": 88, "storage_free_mb": 4096,
    "active_slot": "a" }
}

```

event enumerates the telemetry lifecycle events (master §9): download_started, installing, installed, verifying, success, failure — exactly 6 values (the TelemetryEventType schema). This is **not** the 7-value UpdateState used by §8.2; idle is deliberately excluded because an idle device emits no telemetry event.

- **Response 202 Accepted** (TelemetryAck):

```
{ "accepted": 2, "rejected": 0, "request_id": "01J..." }
```

202 is used because ingestion is asynchronous (events are enqueued to the pipeline). A device **MUST** treat 202 as success and not retry the same batch.

- **Status codes:** 202 Accepted; 400 VALIDATION_FAILED (malformed batch); 401/403 (cannot report for another device); 413 PAYLOAD_TOO_LARGE (oversized batch); 429.

13. Status code summary

Code	Meaning in this API	Where
200	OK (read, login, update available)	most GETs, §7, §12.1
201	Created (registration, upload, release, deployment)	§8.1, §9.1, §10.1, §11.1
202	Accepted (async telemetry ingest)	§12.2
204	No Content (device already up to date)	§12.1

Code	Meaning in this API	Where
206	Partial Content (Range artifact fetch)	artifact byte path (§9.2 note)
400	VALIDATION_FAILED	all write routes
401	UNAUTHENTICATED (missing/invalid/expired token)	all protected routes
403	FORBIDDEN (role/ownership)	all protected routes
404	NOT_FOUND	single-resource reads
409	CONFLICT / VERSION_NOT_MONOTONIC	§8.1, §10.1, §11.1
413	PAYLOAD_TOO_LARGE	§9.1, §12.2
415	UNSUPPORTED_MEDIA_TYPE	§9.1
416	Range Not Satisfiable	artifact byte path
422	HASH_MISMATCH / SIGNATURE_INVALID	§9.1
429	RATE_LIMITED	all rate-limited routes (§5)
500	INTERNAL (no disclosure)	any (via recovery brick)
501	Range unsupported (guarded; must not occur)	artifact byte path

14. Testing (four-layer)

Per master §13 / Constitution §1 (UNVERIFIED clause), every change to this API ships **four layers**, with no-bluff positive evidence (§7.1). Safety-critical paths here — **signature/SHA-256 verification on upload (§9.1)** and **the update-check field contract (§12.1)** — target **≥90% coverage**.

1. **Source-presence gate.** Static assertion that the code artifacts that implement this spec exist: each route handler is registered on the Gin router (`/api/v1/...`), each schema in `ota-protocol` is declared, and the `openapi.yaml` paths match the registered routes one-to-one. Fails if a documented endpoint has no handler, or a handler has no doc/schema.
2. **Artifact gate (bytes shipped).** Assert the *built* binary actually exposes the contract: boot the server, hit `GET /openapi` (or the served spec) and diff the live route table against `openapi.yaml`; assert the `http3` handler is mounted with HTTP/2 fallback and that the artifact path advertises `Accept-Ranges: bytes + Content-Encoding: identity` while JSON routes negotiate `br/gzip`. Confirms the shipped bytes, not just source.
3. **Runtime / integration.** End-to-end happy path against a running monolith + PostgreSQL + MinIO/S3 (containerized dev stack): login → upload (valid sig) → create release → create all-targets deployment → device register → device update-check returns 200 with `{url,offset,size,sha256,signature,payload_properties}` → range-fetch the artifact → telemetry report 202. Negative integration:

update-check returns **204** when on target version; upload of a tampered artifact returns **422 SIGNATURE_INVALID**; wrong-device telemetry returns **403**; Brotli vs gzip vs identity negotiation honored; QUIC→HTTP/2 fallback reachable. (Range-over-HTTP/3 and Android-15 resume headers are UNVERIFIED — closed by the ADR-0004 §6 spike, not asserted here.)

4. **Mutation meta-test (PASS→FAIL on negation).** For the safety-critical checks, mutate the implementation and assert the suite flips PASS→FAIL: e.g. invert the SHA-256 comparison in §9.1 (must fail the upload test), force the update-check to always return **200** (must fail the **204** up-to-date test), drop the signature field from §12.1 (must fail the contract test), or disable RBAC ownership check (must fail the **403** test). A mutation that does not break a test exposes a coverage hole and is itself a defect.

A real Orange Pi 5 Max validation plan
(download→verify→apply→reboot→verify; corrupt-slot → confirm A/B fallback)
is the device-side complement (master §13) and is out of scope for this server-API document.

15. Catalogue-first reuse and decoupling

Per §11.4.74 (catalogue-first, UNVERIFIED clause) and the submodule_reuse_map.md, this API is composed from verified catalogue bricks — **no auth, transport, compression, rate-limiting, or storage logic is hand-rolled:**

Concern	Brick(s)
OAuth2/JWT + RBAC (§4)	auth, security, middleware; KMP Auth-KMP, Security-KMP
Transport HTTP/3→HTTP/2 (§3)	http3
Brotli/gzip negotiation, request-auth, recovery (§3, §6)	middleware, recovery
Rate limiting (§5)	ratelimiter
Artifact blob storage / Range serving (§9, §12.1)	Storage; KMP Storage-KMP
Relational state (devices, releases, deployments)	database
Telemetry ingest / observability (§12.2)	observability, Herald (alerting)
Runtime config (poll interval + jitter, TTLs, limits)	config; KMP Config-KMP
Wire types / schemas	ota-protocol (NEW), ota-telemetry-schema (NEW)
Validation pipeline (§9.1)	ota-artifact-validator (NEW) + security

Decoupling (§11.4.28, UNVERIFIED): transport carries no business logic; the validator has no transport; the (1.0.1) rollout engine is HTTP-free behind the all-targets seam (§11).

16. Compliance notes (HelixConstitution)

Clause numbers are carried from corpus convention and are **UNVERIFIED** against the authoritative Constitution text (the Constitution file is not present in this repository).

Clause (label)	How this spec complies
§11.4.61 (ToC mandatory)	Metadata table first, ToC immediately after, numbered + anchored.
§7.1 / §11.4.6 (anti-bluff / no-guessing)	Every transport/trust claim traces to an ADR or stack note; unconfirmed items (optional AOSP headers, Range-over-HTTP/3, brick public surfaces, concrete rate limits, clause numbers) are marked UNVERIFIED , never invented.
§11.4.74 (catalogue-first reuse)	API composed from verified bricks (§15); no hand-rolled auth/transport/limiter/storage.
§11.4.28 (decoupling)	Single-purpose seams; artifact path separated from JSON control plane; <code>all-targets</code> reserves the rollout seam.
§1 / §1.1 (four-layer + mutation)	§14 defines source-presence → artifact → runtime/integration → mutation meta-test; ≥90% on signing-verify + update-check contract.
LOCKED stack (D6) honored	Go + Gin + Brotli + HTTP/3 via <code>http3</code> submodule with HTTP/2 + gzip fallback; REST primary; gRPC out of scope.
LOCKED strategy honored	Native Android A/B (<code>update_engine</code>) on device + custom Go control plane (modular monolith, ADR-0003); MVP signing + SHA-256 + AVB, TUF device-side deferred to 1.0.1 (ADR-0002); deployments <code>all-targets</code> for MVP.